

ADDIS ABABA UNIVERSITY OF TECHNOLOGY
Department of Software Engineering

GROUP CAPSTONE PROJECT

SECT- 4221 — Enterprise Application Development

Full-Stack Java · Spring Boot 3.x · REST APIs · Security · Deployment

Course	SECT- 4221 — Enterprise Application Development
Academic Year	Spring 2025 / 2026
Group Size	2 – 4 students per group (maximum 4)
Total Weight	20% of final course grade
Format	GitHub repository + deployed application + final presentation
Key Dates	See Phase Timeline on page 2

PURPOSE OF THIS DOCUMENT

This document is your complete guide to the **SECT- 4221** Capstone Project. It contains:

- The project overview, goals, and technical requirements
- The submission process: Phase 1 (Proposal) → Approval → Phase 2 (Development) → Phase 3 (Delivery)
- PART A — the Proposal Template your group submits to get approved
- PART B — the Requirements Document Template you complete after approval
- PART C — the Final Submission checklist and marking rubric

Read the entire document before starting. Groups that skip the approval step will not be marked.

1. Project Overview

1.1 What You Are Building

Your group will design, build, and deploy a **production-quality full-stack web application** using the Java ecosystem covered in SECT- 4221. The application must be a real, useful system — not a tutorial clone or a toy project. It should solve a genuine problem and demonstrate mastery of the full technology stack taught in this course.

The application must include all five pillars of the course:

Pillar	Technology	What is expected
1 Backend API	Spring Boot 3.x + Java 21	REST API with proper HTTP semantics, pagination, error handling
2 Data Layer	Spring Data JPA + PostgreSQL	Normalised schema, Flyway migrations, optimised queries
3 Security	Spring Security 6 + JWT	Authentication, role-based access control, OWASP mitigations
4 Quality	JUnit 5 + Mockito + Testcontainers	Unit, slice, and integration tests — minimum 70% coverage
5 Deployment	Docker + Docker Compose	Containerised app + DB, deployed to cloud (Render / Railway / AWS)

1.2 Application Domain Options

Your group may propose any domain. The following are *suggestions only* — original ideas are encouraged and rewarded:

Domain	Example System	Key Entities
E-Commerce	Online marketplace / auction / rental platform	User, Product, Order, Payment, Review, Inventory
Healthcare	Clinic booking / patient records / pharmacy	Patient, Doctor, Appointment, Prescription, MedRecord
Education	Google Class Room / quiz platform / course marketplace	Student, Instructor, Course, Lesson, Enrolment, Grade
Finance	Budget tracker / loan management / crypto portfolio	Account, Transaction, Category, Budget, Report
Logistics	Fleet management / delivery tracking / warehouse	Vehicle, Driver, Shipment, Route, Warehouse, Customer
HR / Workforce	Leave management / payroll / performance	Employee, Department, Payroll, Leave, Review, KPI
Events	Ticketing / conference management / venue booking	Event, Organiser, Attendee, Ticket, Venue, Schedule

Social / Community	Neighbourhood app / club management / charity	Member, Group, Post, Event, Donation, Notification
--------------------	---	--

1.3 What Makes a Good Project

Good project ✓	Weak project ✗
Original domain or a fresh angle on a known domain	Exact clone of a tutorial (e.g. the standard TODO app)
5+ entities with meaningful relationships	2–3 trivial entities with no real relationships
At least 2 user roles with different access levels	Single admin user with no permission complexity
Business logic that justifies a service layer	Controllers that call repositories directly
Real-world constraints: stock limits, booking conflicts, etc.	No domain constraints — anything can be stored anywhere
Automated tests that prove the system works	Only manual testing via Postman / browser

2. Phase Timeline & Milestones

The project runs over **10 weeks** and is divided into three phases. All deadlines are Friday at 23:59 (SAST).

1 PROPOSAL — Project Title & Requirements Document

Weeks 3 – 5

Week 3	Form your group. Register group members on Google Class Room. One group member opens a GitHub repo named se4801-group-<GroupNumber>.
Week 4	Work on your Proposal (Part A of this document). Submit via Google Class Room by end of Week 4.
Week 5	Lecturer reviews and returns feedback. You receive one of: ✓ Approved / △ Revise & Resubmit / ✗ Rejected.

2 DEVELOPMENT — Build, Test, and Iterate

Weeks 5 – 11

Week 5	After approval: Complete Part B (Requirements Document). Begin development. Set up project structure, Flyway migrations, entity model.
Week 7	Milestone 1 (M1): Entity model + all repositories + service layer skeleton + basic CRUD endpoints working. Push to GitHub.
Week 9	Milestone 2 (M2): Security layer complete (JWT + roles). At least 3 major features complete. Tests passing.
Week 11	Milestone 3 (M3): All features complete. Docker Compose working. Application deployed to cloud. Full test suite green.

3 DELIVERY — Submission & Presentation*Weeks 11 – 12*

Week 11 Fri	Final submission: GitHub repo + deployed URL + PDF report submitted via Google Class Room.
Week 12	Group presentations: 15 minutes demo + 5 minutes Q&A per group. All members must present.

LATE SUBMISSION POLICY

- Milestones (M1, M2): no mark deduction — used for formative feedback only. Missing them loses tutor support.
- Final submission: 10 marks deducted per day late. No submissions accepted after 5 days.
- Presentations: failure to present results in 0 for the presentation component for that member.
- Groups that do not receive Approval before Week 6 may not proceed. No exceptions.

3. Technical Requirements

These are **minimum requirements**. Exceeding them improves your mark.

3.1 Mandatory Technical Stack

Category	Technology	Version / Notes
Language	Java	21 (LTS) — use records, sealed types, text blocks where appropriate
Framework	Spring Boot	3.x — do not use Spring Boot 2.x
Build Tool	Maven	3.9+ — pom.xml must be clean and reproducible
ORM	Spring Data JPA + Hibernate	Entities, relationships, derived queries, JPQL
Database (prod)	PostgreSQL	15+ — via Docker or cloud managed instance
Database (test)	H2 / Testcontainers	H2 for unit tests, Testcontainers for integration
Migrations	Flyway	All schema changes via versioned Vx__name.sql scripts
Security	Spring Security 6 + JWT	Stateless JWT, BCrypt passwords, @PreAuthorize
Testing	JUnit 5 + Mockito	Unit tests + @WebMvcTest + @DataJpaTest + integration tests
Containerisation	Docker + Docker Compose	Single docker-compose.yml spins up app + DB
Version Control	Git + GitHub	Meaningful commit messages; no single 'final' commit
API Documentation	SpringDoc OpenAPI (Swagger UI)	Auto-generated, accessible at /swagger-ui.html
Code Quality	Lombok	Reduce boilerplate; @Data, @Builder, @RequiredArgsConstructor

3.2 Mandatory Functional Requirements

Every project must implement ALL of the following, regardless of domain:

#	Requirement	Why it is required
F1	At least 5 domain entities with JPA relationships	Demonstrates schema design and ORM proficiency
F2	Flyway migrations V1 through V5 minimum	Demonstrates disciplined schema evolution
F3	JWT authentication: register, login, logout with token blacklist	Covers Spring Security + JWT lifecycle
F4	At least 3 user roles with different @PreAuthorize rules	Demonstrates RBAC and method security

F5	At least 8 REST endpoints using proper HTTP semantics	Covers REST design principles
F6	Pagination on at least 2 list endpoints (Spring Pageable)	Demonstrates production-ready API design
F7	Input validation with Bean Validation on all request bodies	@Valid, @NotBlank, @Size, custom validators
F8	Global exception handler returning structured JSON errors	@RestControllerAdvice, consistent error format
F9	At least one search / filter endpoint with multiple parameters	Demonstrates JPA Specifications or JPQL
F10	Full Docker Compose deployment (app + DB + optional Redis)	Containerisation milestone

3.3 Minimum Test Coverage

Layer	Test Type	Minimum Coverage
Service layer	JUnit 5 + Mockito — unit tests for all service methods	Every public method has at least one test
Controller layer	@WebMvcTest — test happy path + error paths per endpoint	All endpoints: 200/201/400/401/403/404
Repository layer	@DataJpaTest or Testcontainers — custom queries only	All derived / custom queries tested
Security rules	MockMvc with @WithMockUser — access control matrix	Every @PreAuthorize rule verified
Overall	JaCoCo coverage report in PDF report	Minimum 70% line coverage

3.4 Architecture Rules

- The application MUST follow the four-layer architecture: Controller → Service → Repository → Entity.
- Controllers must NEVER call repositories directly.
- Entities must NEVER be returned from controllers — always use DTOs.
- Secrets (passwords, JWT secret, API keys) must NEVER be hard-coded — use environment variables.
- Do NOT use `spring.jpa.hibernate.ddl-auto=create-drop` in production configuration. Use Flyway.
- Each class has one responsibility. Services do not contain HTTP logic. Controllers do not contain business logic.

4. Marking Rubric [100 marks total]

Marks are awarded per group, then adjusted for individual contribution via peer review and interview questions during the presentation.

Component	Marks	What the Marker Looks For
Part A — Proposal Document	5	Clear project title, realistic scope, all members listed, requirements are specific and testable
Part B — Requirements Document	10	ER diagram quality, use case completeness, API contract design, security threat model
Architecture & Code Quality	20	4-layer separation, DTOs, no business logic in controllers, clean readable code, no anti-patterns
Data Layer	15	Normalised schema, correct JPA relationships, Flyway migrations, no N+1, proper indexes
REST API Design	15	RESTful URLs, correct HTTP methods & status codes, pagination, validation, global error handler
Security Implementation	15	JWT lifecycle, BCrypt, RBAC, OWASP mitigations, no secrets in code
Testing	10	Coverage $\geq 70\%$, unit + slice + integration + security tests, Testcontainers bonus
Deployment	5	Docker Compose works, application accessible at deployed URL, health endpoint green
Presentation & Demo	5	Clear demo, all members participate, questions answered correctly
TOTAL	100	

INDIVIDUAL ADJUSTMENT

The group receives a single mark. Individual marks are then adjusted based on:

- Git commit history — frequency, quality, and distribution across the codebase
- Peer review score — submitted privately by each member (who contributed what)
- Presentation Q&A — each member will be asked one technical question about the code

A member who made no Git commits will receive 0 regardless of group mark. A member who dominates the codebase while others contribute nothing will also result in adjustments.

5. Approval Process

Step	Who	What happens
1 Submit Proposal	Group → Google Class Room	Complete Part A and submit by end of Week 4. One submission per group.
2 Lecturer Review	Lecturer	Lecturer reviews within 3 business days. Outcome: Approved / Revise & Resubmit / Rejected.
3a APPROVED	Group	You receive a Group Project ID (e.g. GP-2026-07). Begin Part B (Requirements Document) immediately.
3b REVISE & RESUBMIT	Group	You receive written feedback. Resubmit within 3 days. Maximum 2 revision rounds.
3c REJECTED	Group	Scope is not feasible, not original, or not aligned with course. Choose a new topic and resubmit from scratch.
4 Submit Part B	Group → Google Class Room	Submit Requirements Document by end of Week 5. Development may begin once Part B is submitted.
5 Milestone check-ins	Group → GitHub	M1 (Week 7), M2 (Week 9), M3 (Week 11) — push to GitHub. No Google Class Room submission needed for milestones.
6 Final Submission	Group → Google Class Room	Submit GitHub URL + deployed URL + PDF report by Week 11 Friday.
7 Presentation	Group → Venue	15-minute demo + 5-minute Q&A in Week 12. Schedule communicated by Week 10.

COMMON REJECTION REASONS

- Too simple: a CRUD app with 2 entities and no business logic does not meet the course standard.
- Not original: exact clone of the course's ShopWave demo app or any tutorial project found online.
- Out of scope: a mobile app, a machine learning system, or anything that does not use the Spring Boot stack.
- Vague requirements: "users can manage products" is not a requirement. "A SELLER can create a product with a name (max 200 chars), price (>0), and stock (>=0)" is a requirement.
- No security: a project with no authentication layer will always be rejected.

PART A

PROJECT PROPOSAL — Submit by Tuesday

HOW TO USE THIS TEMPLATE

- Delete all grey instruction text before submitting. Your submitted document should contain only your group's content.
- Submit as a PDF (not .docx) via the Google Classroom assignment portal titled: SE4801_GroupXX_Proposal.pdf
- One submission per group. Include all group members' names and student numbers.

A1. Group Information

Group Number	GP-2026-____ (assigned by lecturer)
GitHub Repository URL	https://github.com/HlinaG144/Enterprise-app
Submission Date	

A2. Group Members

#	Full Name	Student Number	Email Address	GitHub Username
1	Hlina Girum	ATE/3417/14	hlinhlina9@gmail.com	HlinaG144
2	Mistre Solomon	ATE/2545/14	Mistre0530@gmail.com	MISTRE21
3	Zerubabel Mesfin	ATE/0211/14	mesfinzeru96@gmail.com	zeruumesfin
4				
5				

Group Leader (point of contact for all lecturer communications): Zerubabel Mesfin

A3. Project Title

Project Title:

Write a clear, descriptive title. Examples: 'CliniBook — GP Clinic Appointment & Records System' or 'FleetSync — Vehicle Tracking and Maintenance Management Platform'

The cinema — Movie Distribution & Rental Management System

A4. One-Line Summary

One-line summary (max 30 words):

A sentence that describes WHO uses the system and WHAT it does. Example: 'A platform where clinic staff book appointments, doctors manage patient records, and administrators generate monthly reports.'

A platform where distributors manage movie licenses, clients rent movies based on active agreements, and administrators oversee rentals, contracts, and system operations.

A5. Problem Statement

Problem Statement (4–8 sentences):

Describe the real-world problem your application solves. Who are the users? What pain point does it address? Why is a software system the right solution? Do not describe the solution here

The movie distribution and rental industry involves multiple stakeholders, including distributors, cinemas, and digital platforms, who must manage movie availability, licensing, and rentals efficiently. Many smaller distributors and rental businesses lack centralized systems to track movie rights, rental status, and client access. This often leads to errors such as unauthorized rentals, expired licenses being ignored, and poor tracking of movie usage. Additionally, managing contracts and monitoring active rentals manually is time-consuming and prone to mistakes. There is a need for a system that ensures proper control over movie distribution, rental workflows, and access rights. A software solution can streamline these operations, improve accuracy, and provide better visibility into business processes.

A6. Application Domain & User Roles

Describe the domain and list ALL user roles. Each role must have a distinct set of responsibilities.

Application Domain (e.g. Healthcare, E-Commerce, HR, Education):

Application Domain:

Entertainment / Media Distribution & Rental

User Roles:

Admin

Manages users, system configuration, and overall operations

Distributor

Uploads movies and creates licensing agreements

Client (Cinema/Platform)

Rents movies based on active licenses

User (Optional – if included)

Views and rents movies (basic access)

Role Name	Description of what this role can do
e.g. ADMIN	Manages system users, generates reports, configures settings
e.g. MANAGER	Creates and approves items, views all records, manages staff
e.g. USER / CUSTOMER	Views own data, creates requests, updates own profile
Role 4 (if applicable)	
Role 5 (if applicable)	

A7. Core Entities (Domain Model Preview)

List at least 5 entities. For each, state the key fields and how it relates to other entities.

Entity Name	Key Fields (name, type)	Relates to
Movie	id, title, genre, releaseDate, duration, availabilityStatus	One Movie → many Licenses & Rentals
Distributor	id, name, email, companyName	One Distributor → many Movies & Licenses
Client	id, name, email, organizationName	One Client → many Rentals
License	id, startDate, endDate, terms, status	Linked to Movie, Linked to Distributor, Assigned to Client
Rental	id, rentalDate, returnDate, status	Linked to Movie, Linked to Client

A8. Top 5 Features

List the five most important features your application will deliver. Be specific — ‘manage products’ is too vague.

#	Feature Name	Short Description (1–2 sentences)
---	--------------	-----------------------------------

1	Movie Catalog Management	Distributors can add, update, and manage movie records
2	License Management System	Create and manage distribution agreements with expiration dates
3	Movie Rental Workflow	Clients can rent movies with status tracking (REQUESTED → ACTIVE → COMPLETED)
4	Role-Based Access Control	Different permissions for Admin, Distributor, and Client
5	Search, Filtering & Reporting	Filter movies by genre, availability, and license status

A9. Technical Scope Confirmation

Confirm that your project will include all mandatory items. Tick (✓) each one:

Mandatory Requirement	Include?	Notes / How you will implement it
5+ JPA entities with relationships	✓	Movie, Distributor, Client, License, Rental
Flyway migrations (V1–V5 minimum)	✓	Will create tables step-by-step
JWT authentication (register, login, logout)	✓	Register, login, logout with blacklist
3+ user roles with @PreAuthorize	✓	ADMIN, DISTRIBUTOR, CLIENT
8+ REST endpoints with correct HTTP semantics	✓	CRUD + search + auth
Pagination on 2+ list endpoints	✓	Movies list, Rentals list
Bean Validation on all request bodies	✓	@NotBlank, @Positive, etc.
Global exception handler (@RestControllerAdvice)	✓	@RestControllerAdvice
Search / filter endpoint with multiple params	✓	Filter by genre, status, license
Docker Compose deployment	✓	App + PostgreSQL

A10. Group Work Plan

Describe how your group plans to divide the work. Who is responsible for which components?

Component	Responsible Member(s)	Estimated Effort
Database schema & Flyway migrations	Mistre and Hlina	Medium
Entity classes & JPA relationships	Zeru	Medium
Repository & service layer	Zeru	High

REST controllers & DTOs	Mistre	Medium
Security (JWT + Spring Security)	Hlina	High
Testing (unit + slice + integration)	Zeru	High
Docker Compose & deployment	Zeru	Medium
PDF report & documentation	Mistre and Hlina	Medium

A11. Risks & Mitigations

Identify 3 risks that could derail your project. How will you handle each?

Risk	Likelihood	Impact	Mitigation Strategy
Complexity of role-based security	Medium	High	Start with basic authentication, then incrementally add roles
Time constraints	High	High	Prioritize core features first, then add enhancements
Integration issues	Medium	Medium	Test each component early and integrate gradually

A12. Declaration

ACADEMIC INTEGRITY DECLARATION

We, the undersigned, declare that:

- This proposal is our original work and has not been submitted for any other course.
- All group members have contributed to and reviewed this proposal.
- We understand that our project will be checked for plagiarism and AI-generation.
- We understand that all group members must be able to explain every part of the final submission.
- We agree to the peer review process and understand that individual marks may differ from the group mark.

Member 1 Signature	Zerubabel Mesfin	Date	22/4/2026
Member 2 Signature	Mistre Solomon	Date	22/4/2026
Member 3 Signature	Hlina Girum	Date	22/4/2026
Member 4 Signature	_____	Date	_____
Member 5 Signature	_____	Date	_____

PART B

REQUIREMENTS DOCUMENT — Submit by Week 2 Friday

HOW TO USE THIS TEMPLATE

- Complete this AFTER your proposal has been approved. Do not submit Part B before receiving approval.
- This document becomes the contract between your group and the lecturer. The final submission will be marked against it.
- If you need to change scope after approval, email the lecturer explaining the change. No undisclosed scope changes accepted.
- Submit as PDF: SE4801_GroupXX_RequirementsDoc.pdf

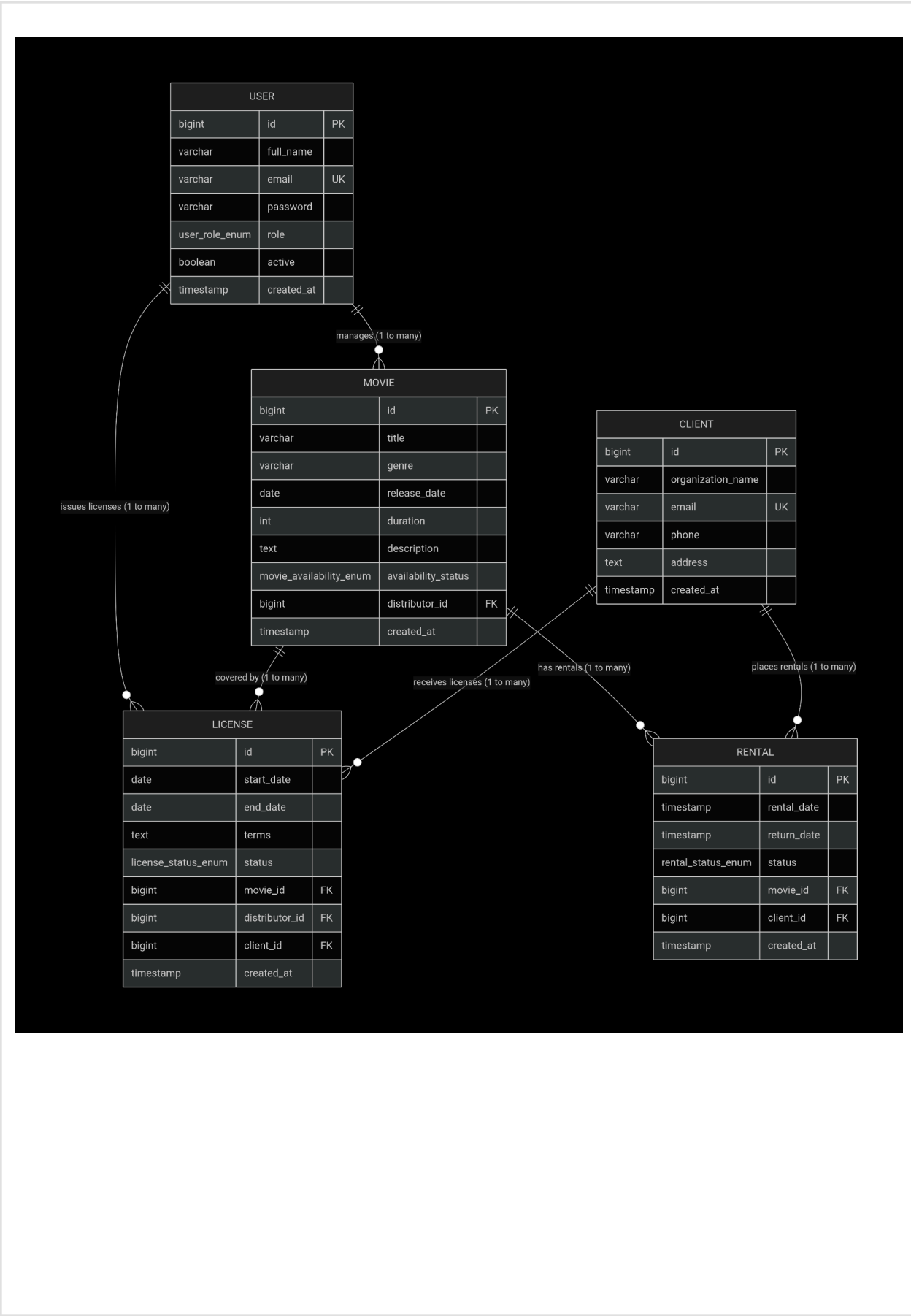
B1. Project Identity

Group Project ID	GP-2026-____ (from approval notification)
Project Title	The cinema
Group Number	
Date Approved	
Requirements Doc Version	v1.0
Deployed URL (update when live)	https://

B2. ER Diagram

ER Diagram (paste image or draw here):

Include ALL entities, attributes (with data types), primary keys, foreign keys, and relationship cardinalities (1:1, 1:N, M:N). Every M:N relationship must be resolved to a join table entity. Use crow's foot or UML notation.



B3. Entity Specifications

Complete one sub-section per entity. Add more sub-sections for additional entities.

B3.1 Entity: user

Field Name	Java Type	Column Name	Constraints	Description
id	Long	id	@Id, @GeneratedValue, PK	Primary key
fullName	String	full_name	@NotBlank, length=100	User full name
email	String	email	UNIQUE, NOT NULL	User email
password	String	password	NOT NULL	BCrypt hashed password
role	Enum(Role)	role	NOT NULL	ADMIN, DISTRIBUTOR, CLIENT
active	Boolean	active	DEFAULT TRUE	Account status
createdAt	LocalDateTime	created_at	@CreationTimestamp, NOT NULL	Auto-set on insert

Relationships:

- User → Rentals : @OneToMany(mappedBy="client")
- User → Movies : @OneToMany(mappedBy="distributor")
- User → Licenses : @OneToMany(mappedBy="distributor")

B3.2 Entity: Movie

Field Name	Java Type	Column Name	Constraints	Description
id	Long	id	@Id, @GeneratedValue, PK	Primary key
title	String	title	NOT NULL	Movie title
genre	String	genre	NOT NULL	Movie categorie
releaseDate	LocalDate	release_date	NOT NULL	Release date
duration	Integer	duration	CHECK >0	Movie duration in minute
description	String	description	TEXT	Movie summary

availabilityStatus	String	availability_status	NOT NULL	AVAILABLE / RENTED
createdAt	LocalDateTime	created_at	@CreationTimestamp	Created date

Relationships:

- Distributor → Movies : @OneToMany(mappedBy="distributor")
- Distributor → Licenses : @OneToMany(mappedBy="distributor")

B3.3 Entity: Distributor

Field Name	Java Type	Column Name	Constraints	Description
id	Long	id	@Id, @GeneratedValue, PK	Primary key
companyName	String	company_name	NOT NULL	Distributor company
email	String	email	UNIQUE	Company email
phone	String	phone	length=20	Contact phone
address	String	address	TEXT	Company address
createdAt	LocalDateTime	created_at	@CreationTimestamp	Created date

Relationships:

- Distributor → Movies : @OneToMany(mappedBy="distributor")
- Distributor → Licenses : @OneToMany(mappedBy="distributor")

B3.4 Entity: Client

Field Name	Java Type	Column Name	Constraints	Description
id	Long	id	@Id, @GeneratedValue, PK	Primary key
organizationName	String	organization_name	NOT NULL	Cinema/platform name
email	String	email	UNIQUE	Client email
phone	String	phone	length=20	Contact phone
address	String	address	TEXT	Client address
createdAt	LocalDateTime	created_at	@CreationTimestamp	Created timestamp

Relationships:

- Client → Rentals : @OneToMany(mappedBy="client")
- Client → Licenses : @OneToMany(mappedBy="client")

B3.5 Entity: License

Field Name	Java Type	Column Name	Constraints	Description
id	Long	id	@Id, @GeneratedValue, PK	Primary key
startDate	LocalDate	start_date	NOT NULL	License start
endDate	LocalDate	end_date	NOT NULL	License expiry
status	String	status	NOT NULL	ACTIVE / EXPIRED
terms	String	terms	TEXT	Agreement terms

Relationships:

- License → Movie : @ManyToOne
- License → Distributor : @ManyToOne
- License → Client : @ManyToOne

B3.6 Entity: Rental

Field Name	Java Type	Column Name	Constraints	Description
id	Long	id	@Id, @GeneratedValue, PK	Primary key
rentalDate	LocalDate	rental_date	NOT NULL	Rental start
returnDate	LocalDate	return_date	NOT NULL	Rental return
status	String	status	NOT NULL	REQUESTED / ACTIVE / COMPLETED
createdAt	LocalDateT ime	created_at	@CreationTimestamp	Created date

Relationships:

- Rental → Movie : @ManyToOne
- Rental → Client : @ManyToOne

B4. Flyway Migration Plan

List each Flyway migration script with its version, filename, and what it does. Minimum 5 versions.

Version	Filename	Description
V1	V1__create_roles_table.sql	Create roles table
V2	V2__create_users_table.sql	Create users table
V3	V3__create_movies_table.sql	Create movies table
V4	V4__create_licenses_table.sql	Create licenses table
V5	V5__create_rentals_table.sql	Create rentals table
V6	V6__insert_default_roles.sql	Insert default roles

B5. API Contract

List every REST endpoint your application will expose. Add rows as needed.

#	Method	Path	Auth Required	Role	Request Body / Params	Response	Description
1	POST	/api/auth/register	No	PUBLIC	RegisterRequest JSON	201 UserDTO	Register user
2	POST	/api/auth/login	No	PUBLIC	LoginRequest JSON	200 TokenResponse	Login user
3	GET	/api/movies	Yes	All	?page&size&sort	200 Page<ProductDTO>	Get all movies
4	PUT	/api/movies/{id}	Yes	DISTRIBUTOR	Path var id	200 / 404	Update movie
5	POST	/api/movies	Yes	ADMIN	CreateProductRequest JSON	201 ProductDTO	Create product
7	DELETE	/api/movies/{id}	Yes	ADMIN	Path var id	204 / 404	Delete movie
8	POST	/api/licenses	Yes	DISTRIBUTOR			Create license
9	GET	/api/licenses	Yes	ADMIN	?keyword&minPrice&maxPrice&category	200 List<DTO>	view licenses
10	POST	/api/rentals	NO	CLIENT			Create rental
11	PATCH	/api/movies/{id}	Yes	DISTRIBUTOR	Status	200	Update rental status

B6. Security Design

B6.1 Authentication Flow

Describe the JWT authentication flow step by step:

Include: register → login → token issued → token sent in header → JwtFilter validates → SecurityContext set → logout/blacklist. Draw a sequence diagram if possible.

1. User registers using `/api/auth/register`
 2. Password is hashed using BCrypt before saving
 3. User logs in via `/api/auth/login`
 4. System validates email and password
 5. JWT token is generated and returned
 6. Client sends JWT in Authorization header: `Bearer <token>`
 7. JwtAuthFilter intercepts requests
 8. Token is validated
 9. SecurityContext is populated
 10. User accesses protected endpoints based on role
 11. On logout, token is added to blacklist
 12. Blacklisted or expired tokens are rejected

B6.2 Role Permission Matrix

List every endpoint and which role can access it. Use ✓ (allowed) or ✗ (denied).

Endpoint	PUBLIC	CLIENT	DISTRIBUTER	ADMIN
GET /api/movies	✓	✓	✓	✓
POST /api/movies	✗	✗	✓	✓
PATCH /api/movies/{id}	✗	✗	✓	✓
DELETE /api/movies/{id}	✗	✗	✗	✓
GET /api/licenses	✗	✗	✓	✓
POST/api/rentals	✗	✓	✗	✗
GET /api/rentals	✗	✓	✗	✓
GET/api/admin/users	✗	✗	✗	✓

B6.3 OWASP Mitigations

For each OWASP API risk, state how your application mitigates it:

OWASP Risk	Your Mitigation Strategy
API1: Broken Object Level Auth (BOLA)	Use <code>@PreAuthorize</code> and ownership validation

API2: Broken Authentication	JWT + BCrypt + token expiration
API3: Broken Object Property Level Auth	DTOs restrict exposed fields
API4: Unrestricted Resource Consumption	Pagination and request limits
API7: SSRF	No external URL fetching from users
API8: Security Misconfiguration	Environment variables + secure Spring config

B7. Non-Functional Requirements

Category	Requirement	How you will achieve it
Performance	Endpoints respond in <300ms	Pagination + indexing
Security	BCrypt hashing	BCryptPasswordEncoder(12)
Security	JWT secret secured	Environment variables
Testability	≥70% coverage	JUnit + Mockito
Deployability	Single docker-compose up	Docker Compose
Observability	Health endpoint	Spring Boot Actuator
Scalability	Support many movie records	Optimized database indexing
Maintainability	Clean layered architecture	Controller-Service-Repository pattern

B8. Test Plan

Describe the tests your group will write for each layer.

Layer	Test Class Name	What it tests	Tools
Service	MovieServiceTest	Create/update/search movies	JUnit 5, Mockito
Service	RentalServiceTest	Rent movie workflow	JUnit 5
Controller	MovieControllerTest	200/201/404 responses	MockMvc
Controller	AuthControllerTest	Login/register validation	@WebMvcTest
Repository	MovieRepositoryTest	Search and filtering queries	@DataJpaTest
Repository	LicenseRepositoryTest	License expiration queries	Testcontainers
Security	SecurityIntegrationTest	Role access control	Spring Security Test
Integration	RentalIntegrationTest	Full rental flow	Testcontainers

B9. Deployment Architecture

Deployment Diagram (paste image or draw here):

Show: browser → Load Balancer / Reverse Proxy → Spring Boot container → PostgreSQL container. Include port numbers, Docker network names, and environment variables. If deploying to cloud, show the cloud service.

Deployment Description

Browser/User

↓

Nginx Reverse Proxy (optional)

↓

Spring Boot App Container (Port 8080)

↓

PostgreSQL Database Container (Port 5432)

Docker Network:

- cine-network

docker-compose.yml service names and ports:

Service Name	Docker Image	Port Mapping	Environment Variables
app	./Dockerfile (build from source)	8080:8080	SPRING_PROFILES_ACTIVE, JWT_SECRET, DB_URL, DB_USER, DB_PASS
db	postgres:15	5432:5432	POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD
redis (optional)	redis:7-alpine	6379:6379	(none required)

PART C

FINAL SUBMISSION CHECKLIST — Week 5 Friday

Before submitting, every item below must be ticked (✓). The marker will use this list as the first pass of the evaluation.

C1. GitHub Repository Checklist

#	Item	Done?
1	Repository is public and accessible at the URL in your submission	
2	README.md contains: project name, description, group members, how to build, how to run, how to run tests, deployed URL	
3	pom.xml is present and the build succeeds with mvn clean package -DskipTests	
4	All src/ code is present — no empty packages, no placeholder TODO classes	
5	Flyway migration scripts are in src/main/resources/db/migration/ (V1 through minimum V5)	
6	application.properties uses environment variables for all secrets (no hardcoded passwords)	
7	docker-compose.yml is present at the repository root and works with a single docker-compose up	
8	Git history has meaningful commits from multiple group members — not a single final commit	
9	JaCoCo coverage report is in target/site/jacoco/ after running mvn test	
10	Swagger UI is accessible at /swagger-ui.html when the app is running	

C2. Code Quality Checklist

#	Item	Done?
1	No @Entity class is returned directly from any @RestController method	
2	No @Repository or @EntityManager is called directly from any @RestController	
3	No business logic (if/else, calculations, validations) exists in @RestController methods	
4	All @Service methods that only read data are annotated @Transactional(readOnly = true)	
5	All passwords are hashed with BCryptPasswordEncoder — no plaintext passwords stored	
6	JWT secret is loaded from an environment variable, not hardcoded in any file	
7	spring.jpa.hibernate.ddl-auto is NOT set to create-drop in the production configuration	
8	No System.out.println() in production code — all logging uses SLF4J	
9	All REST endpoints return ResponseEntity with the correct HTTP status code	

1 0	@Valid annotation is present on all @RequestBody parameters in controllers	
--------	--	--

C3. Feature Completeness Checklist

#	Feature	Implemented?	Endpoint / Class
1	User registration (POST /api/auth/register)		
2	User login returning JWT (POST /api/auth/login)		
3	JWT validation on protected endpoints (JwtAuthFilter)		
4	At least 3 roles with @PreAuthorize rules		
5	BOLA protection on at least 2 resource types		
6	Paginated list endpoint (Pageable + Page<DTO>)		
7	Search/filter endpoint with multiple optional params		
8	Global exception handler (JSON error body)		
9	All 10 mandatory functional requirements (F1–F10 from Section 3)		
1 0	Deployed and accessible at the URL provided		

C4. Testing Checklist

#	Item	Done?
1	At least one unit test per public service method (JUnit 5 + Mockito)	
2	@WebMvcTest: 200, 201, 400, 401, 403, 404 responses tested per controller	
3	@DataJpaTest or Testcontainers: all custom repository queries tested	
4	Security access control matrix tested (at least: unauth → 401, wrong role → 403, BOLA → 403)	
5	All tests pass: mvn test returns BUILD SUCCESS	
6	JaCoCo coverage is ≥70% lines (report in PDF)	

C5. PDF Report Contents

#	Section	Required Content
1	Cover Page	Project title, group number, all member names, GitHub URL, deployed URL
2	ER Diagram	Final ER diagram (updated from Part B if schema changed)
3	API Documentation	List of all endpoints with method, path, auth, request, response (or link to Swagger)
4	Architecture Overview	4-layer diagram with class names; security flow diagram
5	Test Coverage	JaCoCo screenshot showing $\geq 70\%$ line coverage
6	Deployment Evidence	Screenshot of docker-compose up output; screenshot of deployed app
7	Individual Contributions	Table: each member, what they built, Git commit count
8	Peer Review Form	Each member secretly rates others (separate sealed envelope or Google Class Room private submission)
9	Reflection (per member)	Half page: what you learned, what was hardest, what you would do differently

C6. Presentation Guide [Week 12]

Every group member must speak. The demo must show the live deployed application. Plan your 15 minutes as follows:

Time	Who	Content
0:00 – 2:00	Any member	Introduction: problem statement, team, tech stack overview
2:00 – 5:00	Member(s) covering backend	Walk through ER diagram, show Flyway migrations, explain service layer
5:00 – 9:00	Member(s) covering demo	Live demo of the deployed application: register, login, use all major features
9:00 – 12:00	Member(s) covering security & testing	Show JWT flow in Postman; show test run (mvn test); show JaCoCo report
12:00 – 15:00	Any member	Architecture decisions, what went well, what was hardest, lessons learned
15:00 – 20:00	All members	Q&A: each member will be asked at least one technical question about the code they wrote

ACADEMIC INTEGRITY — FINAL SUBMISSION

By submitting this project, all group members declare that:

- All code, documentation, and design artefacts are the original work of the group.
- All use of AI tools (GitHub Copilot, ChatGPT, etc.) has been disclosed in the README and PDF report.
- Every group member understands all parts of the codebase and can explain any section during the Q&A.
- The group understands that identical or near-identical submissions between groups will be flagged and investigated.

Groups found to have plagiarised will receive a mark of 0 for the entire project component, and the matter will be referred to the Academic Integrity Committee.

Appendix A — Recommended Project Structure

This is the recommended package layout for your Spring Boot application. Adapt it to your domain.

```

se4801-group-XX/
├── src/
│   ├── main/
│   │   ├── java/com/groupXX/<yourapp>/
│   │   │   ├── config/
│   │   │   │   ├── SecurityConfig.java           // SecurityFilterChain, CORS
│   │   │   │   └── OpenApiConfig.java           // Swagger / SpringDoc config
│   │   │   ├── controller/
│   │   │   │   ├── AuthController.java          // /api/auth/**
│   │   │   │   ├── ProductController.java       // /api/products/**
│   │   │   │   └── ...                          // one controller per aggregate
│   │   │   ├── service/
│   │   │   │   ├── ProductService.java         // @Service @Transactional
│   │   │   │   └── ...                          // one service per aggregate
│   │   │   ├── repository/
│   │   │   │   ├── ProductRepository.java       // JpaRepository<Product, Long>
│   │   │   │   └── ...
│   │   │   ├── model/ (entities)
│   │   │   │   ├── User.java                    // @Entity
│   │   │   │   ├── Product.java                 // @Entity
│   │   │   │   └── ...
│   │   │   ├── dto/
│   │   │   │   ├── request/ (CreateXxxRequest records)
│   │   │   │   └── response/ (XxxDTO records)
│   │   │   ├── exception/
│   │   │   │   ├── ResourceNotFoundException.java
│   │   │   │   └── GlobalExceptionHandler.java // @RestControllerAdvice
│   │   │   ├── security/
│   │   │   │   ├── JwtUtils.java                // generate, parse, validate
│   │   │   │   ├── JwtAuthFilter.java           // OncePerRequestFilter
│   │   │   │   └── CustomUserDetailsService.java
│   │   │   └── mapper/
│   │   │       └── ProductMapper.java            // entity ↔ DTO conversion
│   │   ├── resources/
│   │   │   ├── db/migration/
│   │   │   │   ├── V1__create_users.sql
│   │   │   │   ├── V2__create_products.sql
│   │   │   │   └── ...
│   │   │   ├── application.properties
│   │   │   └── application-prod.properties // prod overrides
│   │   └── test/java/com/groupXX/<yourapp>/
│   │       ├── service/ProductServiceTest.java
│   │       ├── controller/ProductControllerTest.java
│   │       ├── repository/ProductRepositoryTest.java
│   │       └── security/SecurityIntegrationTest.java
│   ├── Dockerfile
│   ├── docker-compose.yml
│   ├── pom.xml
│   └── README.md

```

Appendix B — Minimum pom.xml Dependencies

Your pom.xml must include at least these dependencies:

```
<dependencies>
  <!-- Web & REST -->
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId></dependency>

  <!-- Security + JWT -->
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId></dependency>
  <dependency><groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId><version>0.12.6</version></dependency>
  <dependency><groupId>io.jsonwebtoken</groupId>

<artifactId>jjwt-impl</artifactId><version>0.12.6</version><scope>runtime</scope></
dependency>

  <!-- Data / JPA / PostgreSQL -->
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId></dependency>
  <dependency><groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId><scope>runtime</scope></dependency>
  <dependency><groupId>org.flywaydb</groupId>
    <artifactId>flyway-core</artifactId></dependency>
  <dependency><groupId>org.flywaydb</groupId>
    <artifactId>flyway-database-postgresql</artifactId></dependency>

  <!-- Validation -->
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId></dependency>

  <!-- Lombok -->
  <dependency><groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId><optional>true</optional></dependency>

  <!-- API Docs -->
  <dependency><groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.5.0</version></dependency>

  <!-- Testing -->
  <dependency><groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-test</artifactId><scope>test</scope></dependency>
  <dependency><groupId>org.springframework.security</groupId>

<artifactId>spring-security-test</artifactId><scope>test</scope></dependency>
  <dependency><groupId>org.testcontainers</groupId>

<artifactId>postgresql</artifactId><version>1.20.0</version><scope>test</scope></de
pendency>

  <!-- H2 (tests only) -->
  <dependency><groupId>com.h2database</groupId>
    <artifactId>h2</artifactId><scope>test</scope></dependency>
</dependencies>

<plugins>
  <!-- JaCoCo for coverage reports -->

<plugin><groupId>org.jacoco</groupId><artifactId>jacoco-maven-plugin</artifactId>
```

```
<version>0.8.12</version>
<executions>
  <execution><goals><goal>prepare-agent</goal></goals></execution>
  <execution><id>report</id><phase>test</phase>
    <goals><goal>report</goal></goals></execution>
</executions>
</plugin>
</plugins>
```

— END OF PROJECT TEMPLATE DOCUMENT —

Questions? Post in the Google Class Room discussion board or email your lecturer. Do not DM — questions benefit everyone.
